

Uniwersytet Wrocławski

Rafał Lewandków

PROGRAMOWANIE URZĄDZEŃ MOBILNYCH 2

WYKŁAD 3

- Korutyny: współbieżność i cykl życia zadań

Zakres, zadanie i wykonanie

Na poprzednim wykładzie dowiedzieliśmy się, że korutyny **„żyją” w CoroutineScope**. **Ustrukturyzowana Współbieżność** to zasada, która mówi, że **cykl życia** korutyny jest **nierozerwalnie związany** z jej zakresem (scope).

Pojęcie	Znaczenie
CoroutineScope	Zakres, w którym uruchamiamy korutyny.
Job	Uchwyt do zadania i jego cyklu życia.
Dispatcher	Określa miejsce wykonywania kodu.
suspend	Pozwala zawiesić wykonanie funkcji.

Samo suspend ani launch nie przenosi pracy na inny wątek.

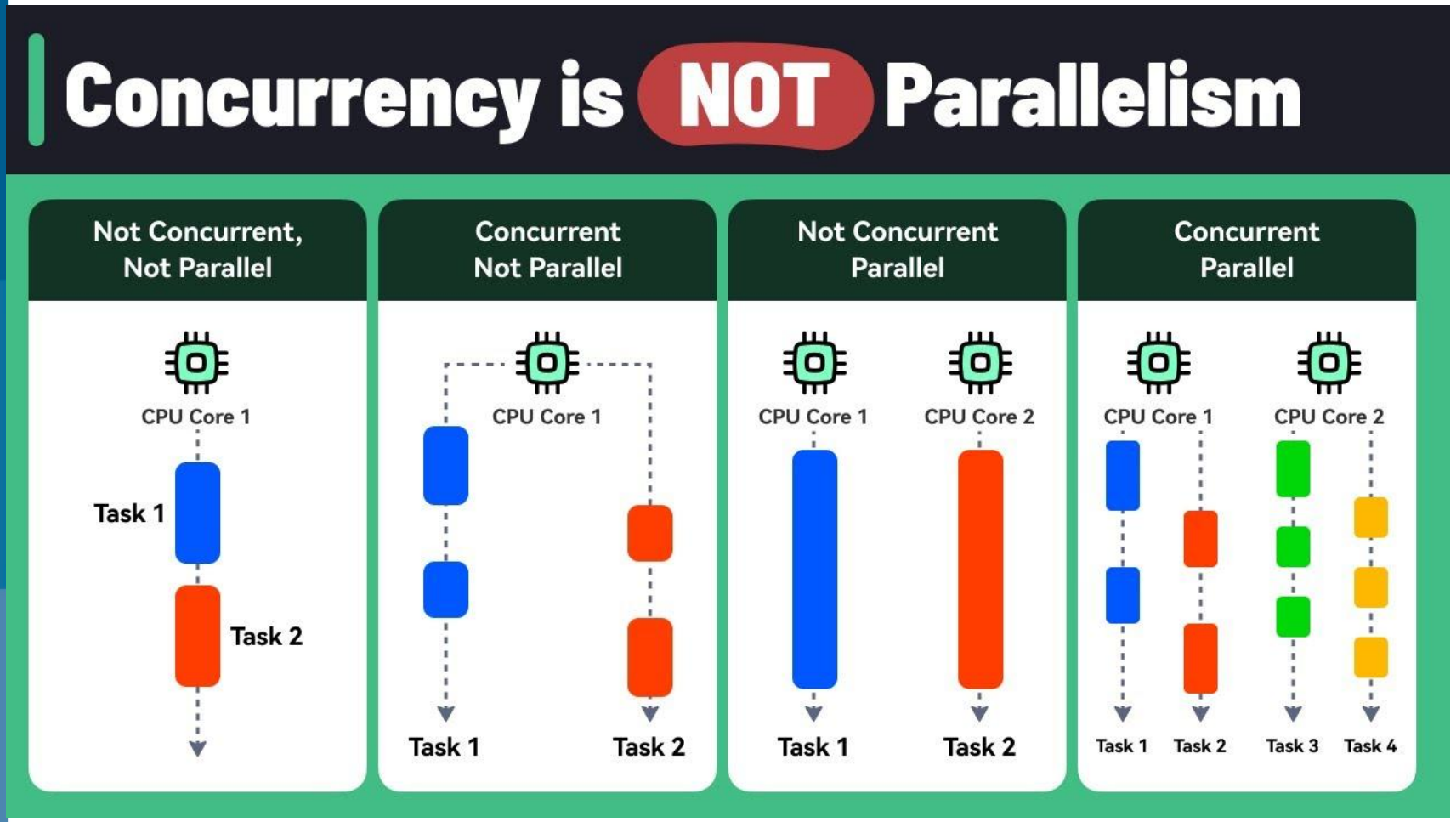
Współbieżność a równoległość

Współbieżne zadania mogą robić postęp na jednym wątku.

Przebieg	Co dzieje się z zadaniami?
Sekwencyjny	Zadanie B rozpoczyna się po zakończeniu A.
Współbieżny, jeden wątek	Gdy A czeka, wątek może wykonywać B.
Równoległy	A i B wykonują kod w tym samym czasie na różnych wątkach.

Współbieżność a równoległość

Współbieżność polega na **zdolności systemu** do obsługi **wielu zadań jednocześnie**, co niekoniecznie oznacza ich wykonywanie w tym samym fizycznym momencie. Chodzi o **zarządzanie wieloma zadaniami** i przełączanie się między nimi tak, aby wszystkie posuwały się do przodu.



https://www.youtube.com/watch?v=RIM9AfWf1WU

Uniwersytet Wrocławski

Ustrukturyzowana współbieżność

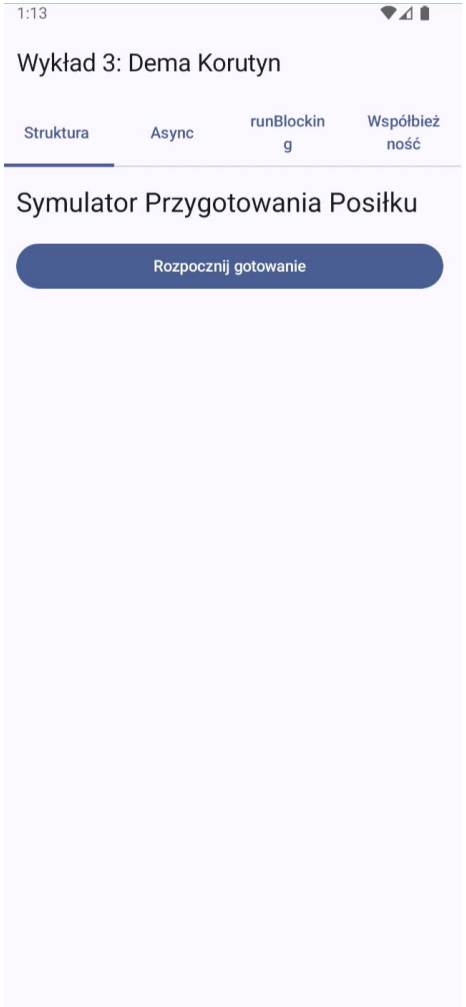
Powiązania rodzic–dziecko tworzą hierarchię zadań.

Zadanie w przykładzie	Rodzic
Przygotowanie posiłku	Job zakresu ekranu Compose
Przygotowanie mięsa	Korutyna przygotowania posiłku
Przygotowanie warzyw	Korutyna przygotowania posiłku

Rodzic kończy się po dzieciach. Anulowanie rodzica obejmuje dzieci.

Przykład: przygotowanie posiłku

Dwa niezależne zadania oraz wspólny komunikat końcowy.



Mięso

Symulowane przygotowanie: 2 s.

Warzywa

Symulowane przygotowanie: 3 s.

Podanie posiłku

Dopiero po zakończeniu obu zadań.

Przy współbieżnym oczekiwaniu całość trwa około 3 s.

Uniwersytet Wrocławski

Stan ekranu i zakres korutyn

Tworzy listę, która przechowuje logi z operacji

Tworzy **bezpieczny** zakres korutyn, **powiązany** z **cyklem życia** tego komponentu.

```
fun MealPrepSimulatorScreen() {
    val logs = remember { mutableStateListOf<String>() }
    val scope = rememberCoroutineScope()

    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp)) {
        Text( text = "Symulator Przygotowania Posiłku", style = MaterialTheme.typography.l
        Spacer(Modifier.height( height = 16.dp))
        Button(onClick = {
            logs.clear()
            scope.launch {
                logs.add("Szef kuchni (rodzic): Zaczynamy!")

                val jobMieso = launch {
                    delay( timeMillis = 2000)
                    logs.add("Kucharz 1: Mięso usmażone (2s).")
                }

                val jobWarzywa = launch {
                    delay( timeMillis = 3000)
                    logs.add("Kucharz 2: Warzywa gotowe (3s).")
                }
                logs.add("Szef kuchni: Zadania zlecone, scope czeka na zakończenie...")

                jobWarzywa.join()
                jobMieso.join()

                logs.add("Szef kuchni: Wszyscy skończyli! Można podawać danie.")
            }
        }, modifier = Modifier.fillMaxWidth()) {
            Text( text = "Rozpocznij gotowanie")
        }
        Spacer(Modifier.height( height = 16.dp))
        LazyColumn {...}
    }
}
```

Uniwersytet Wrocławski

Stan ekranu i zakres korutyn

Tworzy listę, która przechowuje logi z operacji

Tworzy **bezpieczny** zakres korutyn, **powiązany** z **cyklem życia** tego komponentu.

Uruchamia **główną** korutynę.

```
fun MealPrepSimulatorScreen() {
    val logs = remember { mutableStateListOf<String>() }
    val scope = rememberCoroutineScope()

    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp)) {
        Text( text = "Symulator Przygotowania Posiłku", style = MaterialTheme.typography.l
        Spacer(Modifier.height( height = 16.dp))
        Button(onClick = {
            logs.clear()
            scope.launch {
                logs.add("Szef kuchni (rodzic): Zaczynamy!")

                val jobMieso = launch {
                    delay( timeMillis = 2000)
                    logs.add("Kucharz 1: Mięso usmażone (2s).")
                }

                val jobWarzywa = launch {
                    delay( timeMillis = 3000)
                    logs.add("Kucharz 2: Warzywa gotowe (3s).")
                }
                logs.add("Szef kuchni: Zadania zlecone, scope czeka na zakończenie...")

                jobWarzywa.join()
                jobMieso.join()

                logs.add("Szef kuchni: Wszyscy skończyli! Można podawać danie.")
            }
        }, modifier = Modifier.fillMaxWidth()) {
            Text( text = "Rozpocznij gotowanie")
        }
        Spacer(Modifier.height( height = 16.dp))
        LazyColumn {...}
    }
}
```

Uniwersytet Wrocławski

Stan ekranu i zakres korutyn

Tworzy listę, która przechowuje logi z operacji

Tworzy **bezpieczny** zakres korutyn, **powiązany** z **cyklem życia** tego komponentu.

Uruchamia **główną** korutynę.

Uruchamiane są **współbieżne** korutyny. Obie korutyny startują niemal w tym samym momencie i działają **współbieżnie**.

```
fun MealPrepSimulatorScreen() {
    val logs = remember { mutableStateListOf<String>() }
    val scope = rememberCoroutineScope()

    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp)) {
        Text( text = "Symulator Przygotowania Posiłku", style = MaterialTheme.typography.l
        Spacer(Modifier.height( height = 16.dp))
        Button(onClick = {
            logs.clear()
            scope.launch {
                logs.add("Szef kuchni (rodzic): Zaczynamy!")

                val jobMieso = launch {
                    delay( timeMillis = 2000)
                    logs.add("Kucharz 1: Mięso usmażone (2s).")
                }

                val jobWarzywa = launch {
                    delay( timeMillis = 3000)
                    logs.add("Kucharz 2: Warzywa gotowe (3s).")
                }
                logs.add("Szef kuchni: Zadania zlecone, scope czeka na zakończenie...")

                jobWarzywa.join()
                jobMieso.join()

                logs.add("Szef kuchni: Wszyscy skończyli! Można podawać danie.")
            }
        }, modifier = Modifier.fillMaxWidth()) {
            Text( text = "Rozpocznij gotowanie")
        }
        Spacer(Modifier.height( height = 16.dp))
        LazyColumn {...}
    }
}
```

Uniwersytet Wrocławski

Stan ekranu i zakres korutyn

Tworzy listę, która przechowuje logi z operacji

Tworzy **bezpieczny** zakres korutyn, **powiązany** z **cyklem życia** tego komponentu.

Uruchamia **główną** korutynę.

Uruchamiane są **współbieżne** korutyny. Obie korutyny startują niemal w tym samym momencie i działają **współbieżnie**.

Aby wykonać akcję **po zakończeniu wszystkich zadań-dzieci**, musimy na nie **jawnie poczekać.** Służy do tego funkcja join().

```
fun MealPrepSimulatorScreen() {
    val logs = remember { mutableStateListOf<String>() }
    val scope = rememberCoroutineScope()

    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp)) {
        Text( text = "Symulator Przygotowania Posiłku", style = MaterialTheme.typography.l
        Spacer(Modifier.height( height = 16.dp))
        Button(onClick = {
            logs.clear()
            scope.launch {
                logs.add("Szef kuchni (rodzic): Zaczynamy!")

                val jobMieso = launch {
                    delay( timeMillis = 2000)
                    logs.add("Kucharz 1: Mięso usmażone (2s).")
                }

                val jobWarzywa = launch {
                    delay( timeMillis = 3000)
                    logs.add("Kucharz 2: Warzywa gotowe (3s).")
                }
                logs.add("Szef kuchni: Zadania zlecone, scope czeka na zakończenie...")

                jobWarzywa.join()
                jobMieso.join()

                logs.add("Szef kuchni: Wszyscy skończyli! Można podawać danie.")
            }
        }, modifier = Modifier.fillMaxWidth()) {
            Text( text = "Rozpocznij gotowanie")
        }
        Spacer(Modifier.height( height = 16.dp))
        LazyColumn {...}
    }
}
```

Uniwersytet Wrocławski

Stan ekranu i zakres korutyn

Tworzy listę, która przechowuje logi z operacji

Tworzy **bezpieczny** zakres korutyn, **powiązany** z **cyklem życia** tego komponentu.

Uruchamia **główną** korutynę.

Uruchamiane są **współbieżne** korutyny. Obie korutyny startują niemal w tym samym momencie i działają **współbieżnie**.

Aby wykonać akcję **po zakończeniu wszystkich zadań-dzieci**, musimy na nie **jawnie poczekać.** Służy do tego funkcja join().

```
fun MealPrepSimulatorScreen() {
    val logs = remember { mutableStateListOf<String>() }
    val scope = rememberCoroutineScope()

    Column(modifier = Modifier.fillMaxSize().padding( all = 16.dp)) {
        Text( text = "Symulator Przygotowania Posiłku", style = MaterialTheme.typography.l
        Spacer(Modifier.height( height = 16.dp))
        Button(onClick = {
            logs.clear()
            scope.launch {
                logs.add("Szef kuchni (rodzic): Zaczynamy!")

                val jobMieso = launch {
                    delay( timeMillis = 2000)
                    logs.add("Kucharz 1: Mięso usmażone (2s).")
                }

                val jobWarzywa = launch {
                    delay( timeMillis = 3000)
                    logs.add("Kucharz 2: Warzywa gotowe (3s).")
                }
                logs.add("Szef kuchni: Zadania zlecone, scope czeka na zakończenie...")

                jobWarzywa.join()
                jobMieso.join()

                logs.add("Szef kuchni: Wszyscy skończyli! Można podawać danie.")
            }
        }, modifier = Modifier.fillMaxWidth()) {
            Text( text = "Rozpocznij gotowanie")
        }
```

launch zwraca obiekt typu **Job**, który **reprezentuje cykl życia** korutyny.
job.join() to funkcja **suspend**, która **zawiesza korutynę**, w której została wywołana (w tym przypadku rodzica), do czasu, aż **job** się zakończy.

Rodzic uruchamia dwoje dzieci

Kod należy do obsługi kliknięcia przycisku.

```
scope.launch {
    log("Zaczynamy przygotowanie")
    val meat = launch {
        delay(2000)
        log("Mięso gotowe")
    }
    val vegetables = launch {
        delay(3000)
        log("Warzywa gotowe")
    }
    log("Zadania uruchomione")
}
```

Oba launch są wywołane przed oczekiwaniem na zakończenie.

Oczekiwanie przez join()

join() ustala, kiedy wykona się dalszy kod rodzica.

```
scope.launch {
    val meat = launch {
        delay(2000)
        log("Mięso gotowe")
    }
    val vegetables = launch {
        delay(3000)
        log("Warzywa gotowe")
    }
    vegetables.join()
    meat.join()
    log("Można podawać posiłek")
}
```

Dwa join() po kolei nie zmieniają uruchomionych zadań w sekwencyjne.

Rodzic bez jawnego join()

Zakończenie bloku kodu i zakończenie Job to różne momenty.

```
val parent = scope.launch {
    launch { delay(2000); log("Mięso gotowe") }
    launch { delay(3000); log("Warzywa gotowe") }
    log("Koniec bloku rodzica")
}
```

Komunikat

Może pojawić się przed komunikatami dzieci.

Zakończenie Job

Rodzic nadal czeka na zakończenie swoich dzieci.

join() ustala kolejność instrukcji wewnątrz korutyny rodzica.

CoroutineScope i coroutineScope

coroutineScope { ... } grupuje podzadania jednej funkcji suspend.

```
suspend fun prepareMeal(log: (String) -> Unit) {
    coroutineScope {
        launch { delay(2000); log("Mięso gotowe") }
        launch { delay(3000); log("Warzywa gotowe") }
    }
    log("Można podawać posiłek")
}
```

CoroutineScope

Typ zakresu dostępnego np. w scope.launch.

coroutineScope { ... }

Czeka na cały blok i jego dzieci, następnie zwraca wynik.

Po poprawnym zakończeniu coroutineScope oba składniki są gotowe.

Anulowanie całej operacji

Stan isCooking steruje interfejsem, a Job pozwala anulować zadanie.

```
var cookingJob by remember { mutableStateOf<Job?>(null) }
var isCooking by remember { mutableStateOf(false) }

// Kliknięcie „Rozpocznij” (gdy !isCooking):
isCooking = true
cookingJob = scope.launch {
    try {
        prepareMeal(log)
    } finally {
        isCooking = false
    }
}
// Kliknięcie „Anuluj”:
cookingJob?.cancel()
```

Anulowanie rodzica obejmuje obie korutyny przygotowujące składniki.

Asynchroniczność

W poprzednich przykładach włączaliśmy korutyny za pomocą **launch**. W tym przykładzie zobaczymy w jaki sposób odebrać **wynik działania asynchronicznego (async)**.



Asynchroniczność

Funkcja suspend. delay(3000) zawiesza korutynę na 3 sekundy **bez blokowania** wątku UI, a **następnie zwraca wynik** w postaci String.

```
@Composable
fun PasswordGeneratorScreen() {
    var password by remember { mutableStateOf( value = "...") }
    val scope = rememberCoroutineScope()

    suspend fun generatePassword(): String {
        delay( timeMillis = 3000)
        return "Kotlin-Is-Super-Secure-123"
    }

    Column(
        modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text( text = "Generator Tajemniczego Hasła", style = Mater
        Spacer(Modifier.height( height = 20.dp))
        Button(onClick = {
            password = "Generowanie..."
            scope.launch {
                val deferredPassword: Deferred<String> =
                    async { generatePassword() }
                password = deferredPassword.await()
            }
        }) {
            Text( text = "Wygeneruj hasło")
        }
        Spacer(Modifier.height( height = 20.dp))
        Text( text = password, fontSize = 20.sp, fontFamily = Font
    }
}
```

Zwraca wynik

Asynchroniczność

Funkcja suspend. delay(3000) zawiesza korutynę na 3 sekundy **bez blokowania** wątku UI, a **następnie zwraca wynik** w postaci String.

Uruchamia **główną** korutynę

Zwraca wynik

```
@Composable
fun PasswordGeneratorScreen() {
    var password by remember { mutableStateOf( value = "...") }
    val scope = rememberCoroutineScope()

    suspend fun generatePassword(): String {
        delay( timeMillis = 3000)
        return "Kotlin-Is-Super-Secure-123"
    }

    Column(
        modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text( text = "Generator Tajemniczego Hasła", style = Mater
        Spacer(Modifier.height( height = 20.dp))
        Button(onClick = {
            password = "Generowanie..."
            scope.launch {
                val deferredPassword: Deferred<String> =
                    async { generatePassword() }
                password = deferredPassword.await()
            }
        }) {
            Text( text = "Wygeneruj hasło")
        }
        Spacer(Modifier.height( height = 20.dp))
        Text( text = password, fontSize = 20.sp, fontFamily = Font
    }
}
```

Asynchroniczność

Funkcja suspend. delay(3000) zawiesza korutynę na 3 sekundy **bez blokowania** wątku UI, a **następnie zwraca wynik** w postaci String.

Uruchamia **główną** korutynę

Wewnątrz głównej korutyny, async uruchamia nowe, zagnieżdżone zadanie w tle w celu wywołania generatePassword(). Zamiast wyniku, async **natychmiast** zwraca obiekt **Deferred.**

Zwraca wynik

```
@Composable
fun PasswordGeneratorScreen() {
    var password by remember { mutableStateOf( value = "...") }
    val scope = rememberCoroutineScope()

    suspend fun generatePassword(): String {
        delay( timeMillis = 3000)
        return "Kotlin-Is-Super-Secure-123"
    }

    Column(
        modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text( text = "Generator Tajemniczego Hasła", style = Mater
        Spacer(Modifier.height( height = 20.dp))
        Button(onClick = {
            password = "Generowanie..."
            scope.launch {
                val deferredPassword: Deferred<String> =
                    async { generatePassword() }
                password = deferredPassword.await()
            }
        }) {
            Text( text = "Wygeneruj hasło")
        }
        Spacer(Modifier.height( height = 20.dp))
        Text( text = password, fontSize = 20.sp, fontFamily = Font
    }
}
```

Asynchroniczność

Funkcja suspend. delay(3000) zawiesza korutynę na 3 sekundy **bez blokowania** wątku UI, a **następnie zwraca wynik** w postaci String.

Uruchamia **główną** korutynę

Wewnątrz głównej korutyny, async uruchamia nowe, zagnieżdżone zadanie w tle w celu wywołania generatePassword(). Zamiast wyniku, async **natychmiast** zwraca obiekt **Deferred.**

Deferred jest to obiekt, który otrzymujesz **natychmiast po wywołaniu** async, zanim jeszcze właściwe obliczenia się zakończą. Podobnie jak Job, Deferred również **reprezentuje cykl życia** zadania.

Zwraca wynik

```
@Composable
fun PasswordGeneratorScreen() {
    var password by remember { mutableStateOf( value = "...") }
    val scope = rememberCoroutineScope()

    suspend fun generatePassword(): String {
        delay( timeMillis = 3000)
        return "Kotlin-Is-Super-Secure-123"
    }

    Column(
        modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text( text = "Generator Tajemniczego Hasła", style = Mater
        Spacer(Modifier.height( height = 20.dp))
        Button(onClick = {
            password = "Generowanie..."
            scope.launch {
                val deferredPassword: Deferred<String> =
                    async { generatePassword() }
                password = deferredPassword.await()
            }
        }) {
            Text( text = "Wygeneruj hasło")
        }
        Spacer(Modifier.height( height = 20.dp))
        Text( text = password, fontSize = 20.sp, fontFamily = Font
    }
}
```

Asynchroniczność

Funkcja suspend. delay(3000) zawiesza korutynę na 3 sekundy **bez blokowania** wątku UI, a **następnie zwraca wynik** w postaci String.

Uruchamia **główną** korutynę

Wewnątrz głównej korutyny, async uruchamia nowe, zagnieżdżone zadanie w tle w celu wywołania generatePassword(). Zamiast wyniku, async **natychmiast** zwraca obiekt **Deferred.**

Deferred jest to obiekt, który otrzymujesz **natychmiast po wywołaniu** async, zanim jeszcze właściwe obliczenia się zakończą. Podobnie jak Job, Deferred również **reprezentuje cykl życia** zadania.

Korutyna launch **zawiesza się** w tym miejscu, czekając na zakończenie zadania. Gdy async zakończy pracę i dostarczy wynik, await() **"odpakowuje"** go i **przypisuje do zmiennej stanu password**, co powoduje finalną **aktualizację UI (rekompozycję).**

```
@Composable
fun PasswordGeneratorScreen() {
    var password by remember { mutableStateOf( value = "...") }
    val scope = rememberCoroutineScope()

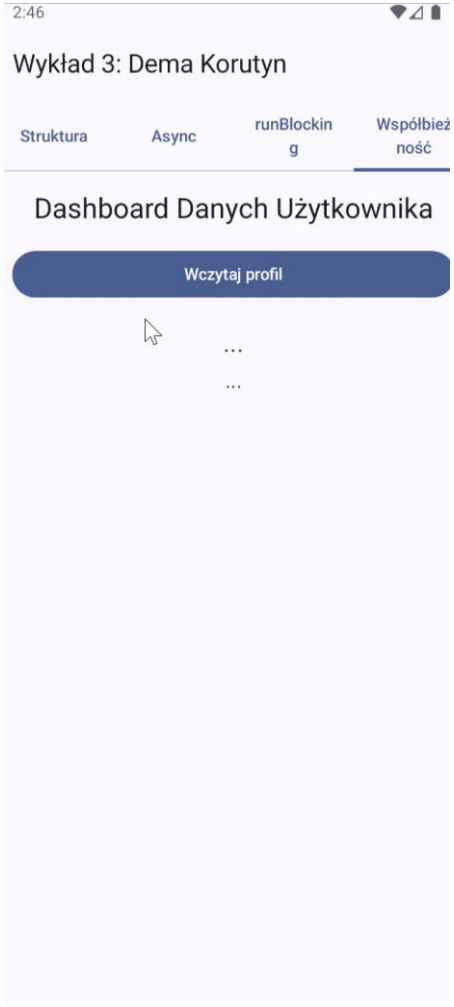
    suspend fun generatePassword(): String {
        delay( timeMillis = 3000)
        return "Kotlin-Is-Super-Secure-123"
    }

    Column(
        modifier = Modifier.fillMaxSize().padding( all = 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text( text = "Generator Tajemniczego Hasła", style = Mater
        Spacer(Modifier.height( height = 20.dp))
        Button(onClick = {
            password = "Generowanie..."
            scope.launch {
                val deferredPassword: Deferred<String> =
                    async { generatePassword() }
                password = deferredPassword.await()
            }
        }) {
            Text( text = "Wygeneruj hasło")
        }
        Spacer(Modifier.height( height = 20.dp))
        Text( text = password, fontSize = 20.sp, fontFamily = Font
    }
```

Zwraca wynik

Przykład: profil użytkownika

Ekran potrzebuje dwóch niezależnych informacji.



Nazwa użytkownika
Symulowane pobieranie trwa 2 s.

Ostatnia aktywność
Symulowane pobieranie trwa 3 s.

Wspólny wynik
Pokazujemy komplet danych i zmierzony czas.

Najpierw porównamy wywołania sekwencyjne z dwoma async.

Funkcje zwracające dane

delay() symuluje oczekiwanie na dane. Funkcje zwracają zwykłe wartości.

```
data class Profile(val name: String, val activity: String)

suspend fun fetchUserName(): String {
    delay(2000)
    return "Ada"
}

suspend fun fetchUserActivity(): String {
    delay(3000)
    return "Dodała post"
}
```

Dane pobierane sekwencyjnie

Kolejne wywołanie zaczyna się dopiero po zakończeniu poprzedniego.

```
suspend fun loadProfileSequential(): Profile {
    val name = fetchUserName()
    val activity = fetchUserActivity()
    return Profile(name, activity)
}
```

Pierwszy wynik

Nazwa użytkownika jest dostępna po około 2 s.

Kompletny profil

Aktywność wymaga następnych około 3 s.

Łącznie około 5 s. Interfejs może nadal reagować dzięki delay().

async rozpoczyna podzadanie

Najpierw uruchamiamy obie operacje, a dopiero potem odbieramy wyniki.

```
suspend fun loadProfile(): Profile = coroutineScope {
    val nameTask = async { fetchUserName() }
    val activityTask = async { fetchUserActivity() }

    Profile(nameTask.await(), activityTask.await())
}
```

Wartość nameTask

Deferred<String>: uchwyt do zadania i przyszłego wyniku.

Kontekst wykonania

async domyślnie korzysta z kontekstu otaczającego zakresu.

async tworzy korutynę potomną.

await() odbiera wynik

await() zawiesza wywołującą korutynę, jeżeli wynik nie jest jeszcze gotowy.

```
suspend fun loadProfile(): Profile = coroutineScope {
    val nameTask = async { fetchUserName() }
    val activityTask = async { fetchUserActivity() }

    val name = nameTask.await()
    val activity = activityTask.await()
    Profile(name, activity)
}
```

Wynik jest dostępny po około 3 s. await() może też zgłosić wyjątek.

launch, async, join i await

Każda z tych operacji ma inną rolę.

Operacja	Co otrzymujemy?	Oczekiwanie
launch { ... }	Job	job.join()
async { ... }	Deferred<T>	task.await()
job.join()	Unit	Czeka na zakończenie zadania.
task.await()	T albo wyjątek	Odbiera wynik zadania.

Deferred<T> jest także Job: można sprawdzić jego stan i go anulować.

Jeden wynik z funkcji suspend

Dla jednej operacji wystarczy bezpośrednie wywołanie w korutynie.

Bezpośredni odbiór wyniku

```
scope.launch {
    val name = fetchUserName()
    log(name)
}
```

Dodatkowa korutyna bez korzyści w tym przykładzie

```
scope.launch {
    val task = async { fetchUserName() }
    log(task.await())
}
```

async przydaje się, gdy chcemy uruchomić pracę współbieżnie.

Błąd jednego z dzieci

W zwykłym coroutineScope niepowodzenie dziecka kończy wspólną operację.

```
suspend fun loadWithFailure(): Profile = coroutineScope {
    val nameTask = async { fetchUserName() }
    val activityTask = async<String> {
        delay(1000)
        throw IOException("Brak połączenia")
    }
    Profile(nameTask.await(), activityTask.await())
}
```

Po około 1 s

activityTask zgłasza błąd. nameTask zostaje anulowane.

Wywołujący

Otrzymuje wyjątek zamiast częściowego profilu.

Obsługę błędu umieszczamy wokół wywołania całej operacji.

Miejsce wykonywania kodu

Dispatcher wybieramy dla rodzaju wykonywanej pracy.

```
scope.launch {
    val sorted = withContext(Dispatchers.Default) {
        List(200_000) { index -> 200_000 - index }
            .sorted()
    }
    log("Posortowano ${sorted.size} liczb")
}
```

Dispatchers.Default

Obliczenia intensywnie wykorzystujące CPU.

Dispatchers.IO

Blokujące operacje wejścia–wyjścia.

withContext zwraca wynik i po zakończeniu przywraca kontekst wywołujący.